

Explainable AI Code Generation Assistant

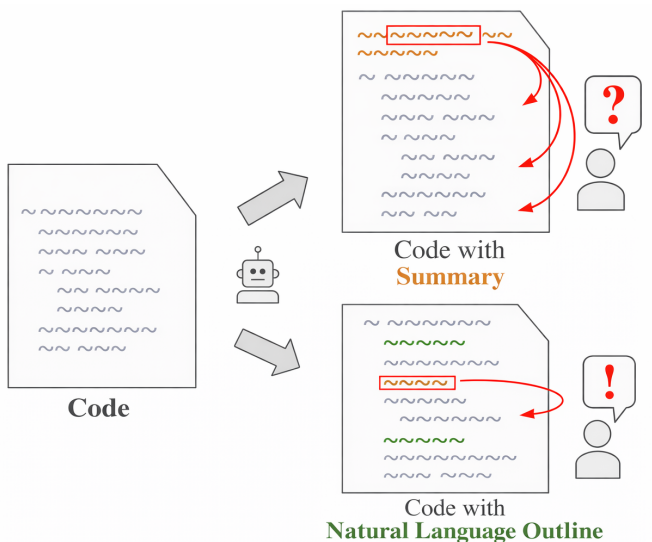
Author: Iuliia Liutova

Supervisor: Alexander Avdiushenko

Neapolis University Paphos

Problem

- ▶ LLMs write code, but do not explain decisions clearly.
- ▶ No mapping between summary and code.



Relevance

Researchers from Meta and academic collaborators introduced a framework for scaling test-time compute in long-horizon coding agents.

They transitioned away from using raw execution trajectories in favor of **structured summaries**.

Scaling Test-Time Compute for Agentic Coding

arXiv, 16 Apr 2026

<https://arxiv.org/abs/2604.16529>

Goal

Build a code assistant that **generates code and human-friendly explanations** aligned with the code.

☒ Structured Granularity ☐

The function filters a list of users to find those who are currently active and have been members for more than a year. It then formats each qualifying user's name and email into a string. The result is a list of these strings.

```
1 def process_user_data(users):
2     active_users = [u for u in users if u['status'] == 'active']
3
4     from datetime import datetime, timedelta
5     one_year_ago = datetime.now() - timedelta(days=365)
6     long_term_users = [
7         u for u in active_users if u['join_date'] < one_year_ago
8     ]
9
10    results = []
11    for u in long_term_users:
12        results.append(f"{u['name']} <{u['email']}>")
13    return results
```

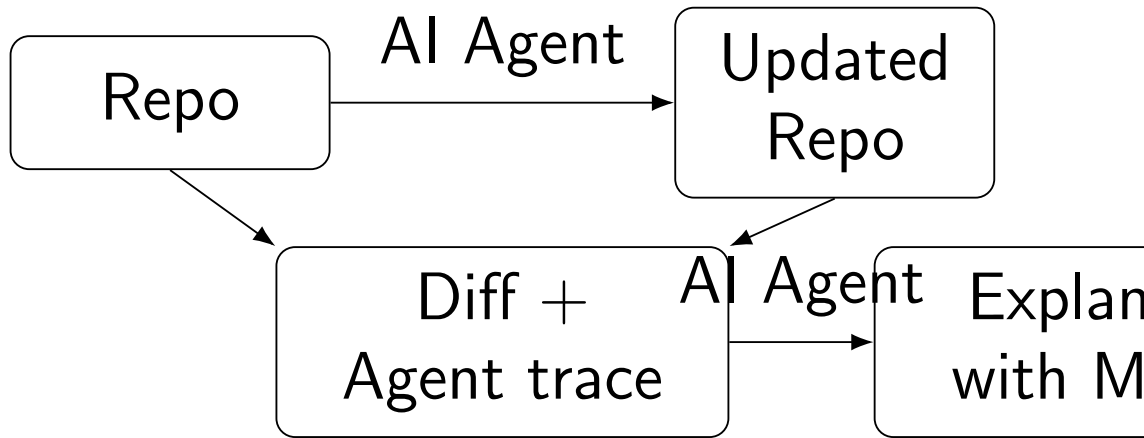
Moves cursor to the next segment

☒ Structured Granularity ☐

The function filters a list of users to find those who are currently active and have been members for more than a year. It then formats each qualifying user's name and email into a string. The result is a list of these strings.

```
1 def process_user_data(users):
2     active_users = [u for u in users if u['status'] == 'active']
3
4     from datetime import datetime, timedelta
5     one_year_ago = datetime.now() - timedelta(days=365)
6     long_term_users = [
7         u for u in active_users if u['join_date'] < one_year_ago
8     ]
9
10    results = []
11    for u in long_term_users:
12        results.append(f"{u['name']} <{u['email']}>")
13    return results
```

High-level architecture



Code with Explanations

Video demo is available in
`video_demo.mp4`

Open the exported HTML on GitHub Pages to play it inline.

SWE-bench Verified Mini: Django subset

What is in the dataset

- ▶ Real bug-fix tasks from django/django.
- ▶ For each task: GitHub issue text, starting commit, Django version, and setup commit.
- ▶ Reference solution: gold code patch and test patch.
- ▶ Evaluation tests: FAIL_TO_PASS checks fixed by the patch and PASS_TO_PASS regression checks.

Why it is needed

- ▶ Gives realistic repository-level tasks without running the full SWE-bench Verified set.
- ▶ Keeps evaluation focused on one codebase, so results are easier to compare and explain.
- ▶ Provides ground truth patches and tests for measuring whether generated code and explanations match the real fix.

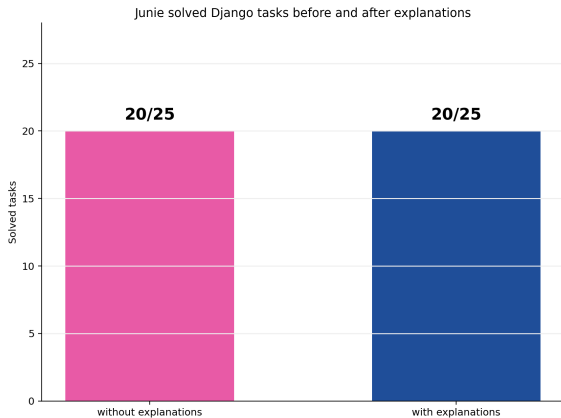
Evaluation Plan

1. Run each Django task twice: baseline agent and agent with explanations.
2. Check correctness with the dataset test patch and existing regression tests.
3. Compare solved tasks: PASS/FAIL before and after explanations.
4. Compare agent efficiency: number of actions on tasks with passed evaluation.
5. Review explanations for grounding: do they describe the actual code changes?

Explanations: Junie gemini-3.1-flash-lite-preview

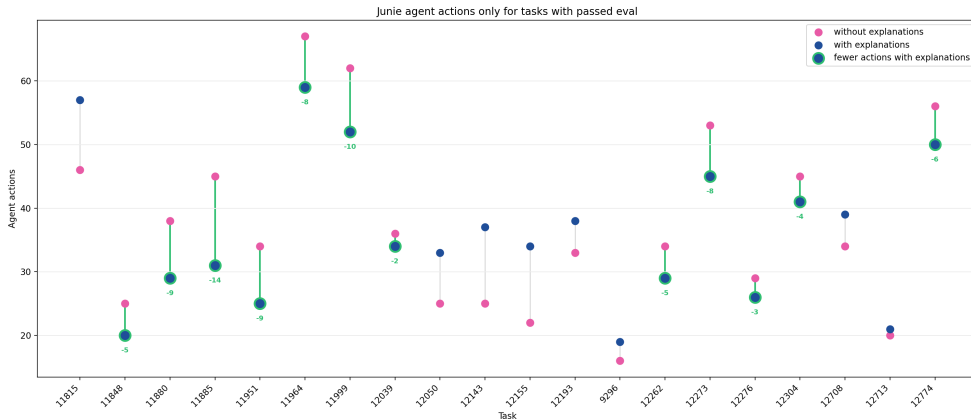
Code generation: Junie gemini-3-flash-preview

Results



Solved Django tasks before and after adding explanations.

Results



Agent action counts on Django tasks with passed evaluation.

Summary

- ▶ Built a prototype for code generation with aligned explanations.
- ▶ Prepared a Django-only evaluation setup using SWE-bench Verified Mini.
- ▶ Initial result: solved tasks stayed at 20/25 after adding explanations.
- ▶ On passed tasks, explanations reduced Junie's actions in 12/20 cases.

Next: evaluate explanation grounding and extend experiments to more repositories and agents.